

TicketForge Architecture

الخطوة الأولى: تصميم الـ High-Level Architecture ومسار البيانات (Data Flow)

1. المقدمة والفلسفة الهندسية (The Architecture Philosophy)

عند تصميم نظام حجز تذاكر عالي التزامن (**High-Concurrency**)، التحدي الأكبر ليس كتابة الكود البرمجي، بل في كيفية حماية مصادر النظام (قواعد البيانات و**Memory**) من الانهيار المفاجئ عند تدفق آلاف الطلبات في نفس الثانية (**Flash Traffic**).

تعتمد هندسة **TicketForge** على مبادئ أساسيين من فلسفة الـ DevOps والأنظمة الموزعة:

- **(فك الارتباط) Decoupling**: فصل الخدمات عن بعضها عبر وسيط رسائل (**Message Broker**) بحيث لا تضطر الخدمات لانتظار بعضها البعض (**Asynchronous Communication**).
- **In-Memory First Validation**: التحقق من البيانات وتأمين الأقفال على مستوى الذاكرة السريعة قبل إرهاق قاعدة البيانات الصلبة.

التحدي التقني المصاحب (Race Conditions)

إذا حاول N من المستخدمين (حيث $N > 1000$) حجز المقعد نفسه في نفس الأجزاء من الثانية، فإن الاعتماد على قواعد البيانات التقليدية سيؤدي حتماً إلى حدوث بيع مزدوج أو بطء شديد (**Deadlock**). الحل يكمن في مسار البيانات الذكي التالي.

2. تفاصيل مسار البيانات اللحظي (Step-by-Step Data Flow)

يمر الطلب (**Request**) المرسل من المستخدم عبر المراحل التالية لضمان الحماية والكفاءة:

01 بوابة الحماية والتوجيه (API Gateway)

يستقبل الـ API Gateway الطلب أولاً، ويقوم بالتحقق من الهوية (**JWT Validation**) وتطبيق حدود الطلبات (**Rate Limiting**) لمنع الهجمات والـ Bots.

02 التحقق السريع عبر الذاكرة (Distributed Locking with Redis)

تمرر البوابة الطلب إلى `ticket-service` لا يذهب السيرفيس إلى قاعدة البيانات، بل يطلب قفلاً مؤقتاً من **Redis** للمقعد المطلوب. إذا كان المقعد محجوزاً مؤقتاً في **Redis**، يتم رفض الطلب فوراً في غضون ميكروثانية دون المساس بـ **DB**.

03 الحجز المبدئي المستدام (State Persistence)

إذا نجح السيرفيس في الحصول على القفل من Redis، يقوم بإنشاء سجل حجز في قاعدة بيانات PostgreSQL وتكون حالته `PENDING_PAYMENT` مع تحديد وقت انتهاء (مثلاً 5 دقائق).

إطلاق الحدث (Asynchronous Event Ingestion)

04

فور حفظ الحجز المبدئي، يقوم `ticket-service` بنشر حدث (*Event*) في **Apache Kafka** ضمن الـ Topic المسمى `ticket.reserved`، وينتهي دور السيرفيس هنا ليصبح جاهزاً لاستقبال طلب جديد فوراً.

معالجة الدفع المعزولة (Payment Simulation)

05

يتلقى `payment-service` الحدث من كافكا بشكل آلي، ويتواصل مع نظام البنك (وهمي). عند اكتمال الدفع، يرسل حدثاً جديداً لكافكا باسم `payment.confirmed`.

توليد المستندات والإخطار (Heavy Background Workers)

06

يتلقى `notification-service` حدث التأكيد، ويقوم بالعمليات المستهلكة للمعالج (*CPU-Intensive*) مثل توليد ملف الـ PDF للتذكرة وإرسال الإيميل للمستخدم دون التأثير على سرعة حجز التذاكر الأساسية.

3. توزيع المسؤوليات والتقنيات (Microservices Architecture Breakdown)

يتكون النظام من أربع خدمات مستقلة تماماً، مجهزة لتعمل داخل حاويات *Docker* وبيئة *Kubernetes* لاحقاً:

اسم الخدمة (Service)	البيئة التقنية (Tech Stack)	الدور والمسؤولية التقنية
api-gateway	Spring Cloud Gateway / Custom	نقطة الدخول الموحدة، معالجة الـ Routing، وعمل الـ Rate Limiting عبر Redis.
ticket-service	Java Spring Boot + PostgreSQL	إدارة العمليات الحساسة (<i>ACID Transactions</i>)، تأمين الأفعال الموزعة، وحفظ الحجوزات.
payment-service	Java / Python FastAPI	الاستماع لأحداث الحجز، معالجة معاملات الدفع، وإخطار النظام بالنتيجة عبر كافكا.
notification-service	Python FastAPI	خدمة خلفية (<i>Worker</i>) لتوليد التذاكر بصيغة PDF وإرسال البريد الإلكتروني بشكل غير متزامن.

4. رؤية الـ DevOps للمشروع (The Production Checklist)

الهدف الأسمى من هذا التصميم ليس مجرد تشغيل الكود، بل تطبيق المفاهيم التالية في الخطوات القادمة:

- Metrics & Observability:** مراقبة كمية الرسائل في Kafka وفترات الاستجابة في Redis باستخدام Prometheus و Grafana.
- Horizontal Pod Autoscaling (HPA):** ضبط Kubernetes لزيادة عدد حاويات `ticket-service` ديناميكياً عند زيادة الطلبات.
- Idempotency:** التأكد من أن إعادة إرسال الرسالة في كافكا عند حدوث خطأ في الشبكة لا تؤدي إلى سحب الأموال مرتين من المستخدم.